

## Data Science:

### Data Loading & Initial Stat :-

- **Pandas (pd):** Reads dataset files and structures data into rows and columns
- **Matplotlib & Seaborn:** Libraries for creating data visualizations and graphs

CODE :

```
import matplotlib.pyplot as plt
```

```
import seaborn as sns
```

```
from sklearn.preprocessing import LabelEncoder
```

```
data = pd.read_csv("/content/iris_1.csv")
```

```
print("Initial Data Preview:")
```

```
display(data.head())
```

```
print("\nNull Values Before Cleaning:")
```

```
display(data.isnull().sum())
```

OUTPUT :

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import LabelEncoder

data = pd.read_csv("/content/iris_1.csv")

print("Initial Data Preview:")
display(data.head())

print("\nNull Values Before Cleaning:")
display(data.isnull().sum())
```

Initial Data Preview:

|   | SepalLengthCm | SepalWidthCm | PetalLengthCm | PetalWidthCm | Species     |
|---|---------------|--------------|---------------|--------------|-------------|
| 0 | NaN           | NaN          | NaN           | 0.2          | Iris-setosa |
| 1 | NaN           | NaN          | NaN           | 0.2          | Iris-setosa |
| 2 | NaN           | 3.2          | NaN           | 0.2          | Iris-setosa |
| 3 | NaN           | 3.1          | NaN           | 0.2          | Iris-setosa |
| 4 | 5.0           | 3.6          | 1.4           | 0.2          | Iris-setosa |

Null Values Before Cleaning:

|               |   |
|---------------|---|
| SepalLengthCm | 4 |
| SepalWidthCm  | 2 |
| PetalLengthCm | 4 |
| PetalWidthCm  | 0 |
| Species       | 0 |

dtype: int64

## Data Cleaning :-

- Imputation: Replacing missing values (NaN) with Mean or Median to avoid deleting valuable row data

CODE:

- Impute missing values with Mean and Median

```
data['SepalLengthCm'] =  
data['SepalLengthCm'].fillna(data['SepalLengthCm'].mean())  
  
data['SepalWidthCm'] =  
data['SepalWidthCm'].fillna(data['SepalWidthCm'].median())  
  
data['PetalLengthCm'] =  
data['PetalLengthCm'].fillna(data['PetalLengthCm'].median())
```

- Drop any remaining nulls

```
data = data.dropna()  
  
print("Null values after cleaning:")  
  
display(data.isnull().sum())
```

OUTPUT :

```
data['SepalLengthCm'] = data['SepalLengthCm'].fillna(data['SepalLengthCm'].mean())  
data['SepalWidthCm'] = data['SepalWidthCm'].fillna(data['SepalWidthCm'].median())  
data['PetalLengthCm'] = data['PetalLengthCm'].fillna(data['PetalLengthCm'].median())  
  
data = data.dropna()  
  
print("Null values after cleaning:")  
display(data.isnull().sum())
```

... Null values after cleaning:  
0  
SepalLengthCm 0  
SepalWidthCm 0  
PetalLengthCm 0  
PetalWidthCm 0  
Species 0  
dtype: int64

# Matplotlib Visualizations:-

## Key Concepts:

- **Line Plot:** Shows the upward or downward trend of a single numerical column
- **Histogram:** Shows the frequency of data points grouped into continuous ranges (bins)
- **Pie Chart:** Displays the percentage distribution of categorical data (requires `.value_counts()` first)
- **Box Plot:** Shows the data's median, middle 50% spread, and extreme outliers (dots outside the lines)

## Code:

### # Line Plot

```
plt.plot(data['SepalLengthCm'])  
plt.title('Line Plot - Sepal Length')  
plt.show()
```

### # Histogram

```
plt.hist(data['PetalLengthCm'], bins=20)  
plt.title('Histogram - Petal Length')  
plt.show()
```

### # Pie Chart

```
species_counts = data['Species'].value_counts()  
species_counts.plot(kind='pie', autopct='%1.1f%%')
```

```
plt.title('Species Distribution')
```

```
plt.ylabel('')
```

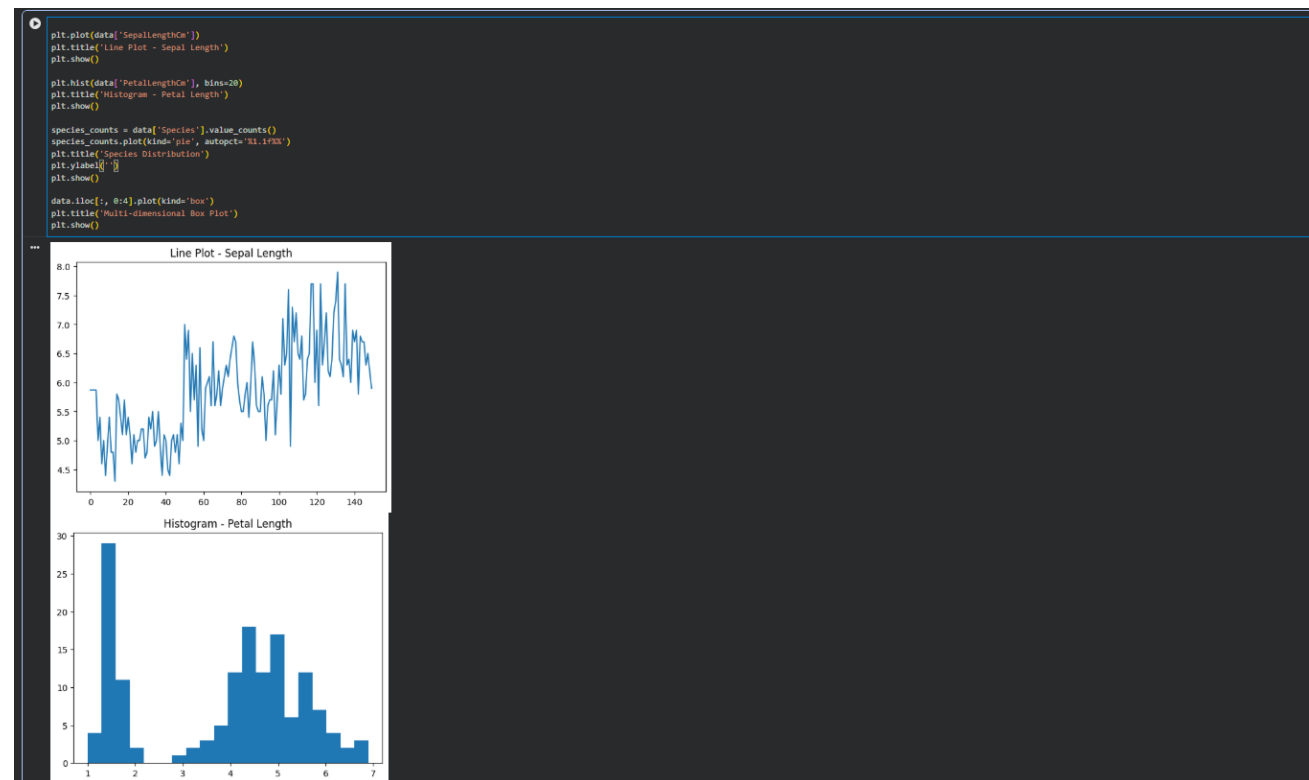
```
plt.show()
```

```
# Multi-Dimensional Box Plot
```

```
data.iloc[:, 0:4].plot(kind='box')
```

```
plt.title('Multi-dimensional Box Plot')
```

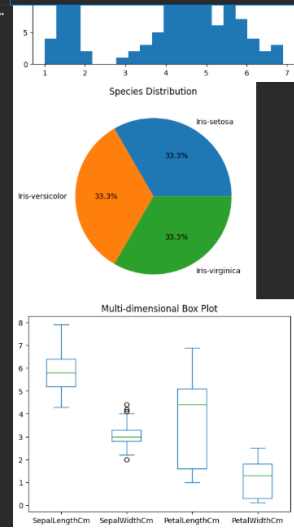
```
plt.show()
```



```
plt.hist(data['Petal.Length'], bins=20)
plt.title('Histogram - Petal.Length')
plt.show()

species_counts = data['Species'].value_counts()
species_counts.plot(kind='pie', autopct='%1.1f%%')
plt.title('Species Distribution')
plt.ylabel('')
plt.show()

data.iloc[:, 0:4].plot(kind='box')
plt.title('Multi-dimensional Box Plot')
plt.show()
```



# Seaborn Visualizations & Heatmap:-

## Key Concepts:

- **Violin Plot:** Combines a box plot with a density curve to show where data values are most heavily concentrated
- **Scatter Plot (hue):** Plots two features against each other and automatically color-codes the dots by category
- **Heatmap:** Uses a color matrix to show correlation strength (values close to 1.0 or -1.0 indicate strong relationships)

## Code:

### # Violin Plot

```
sns.violinplot(x='Species', y='SepalLengthCm', data=data)
plt.title('Seaborn Violin Plot')
plt.show()
```

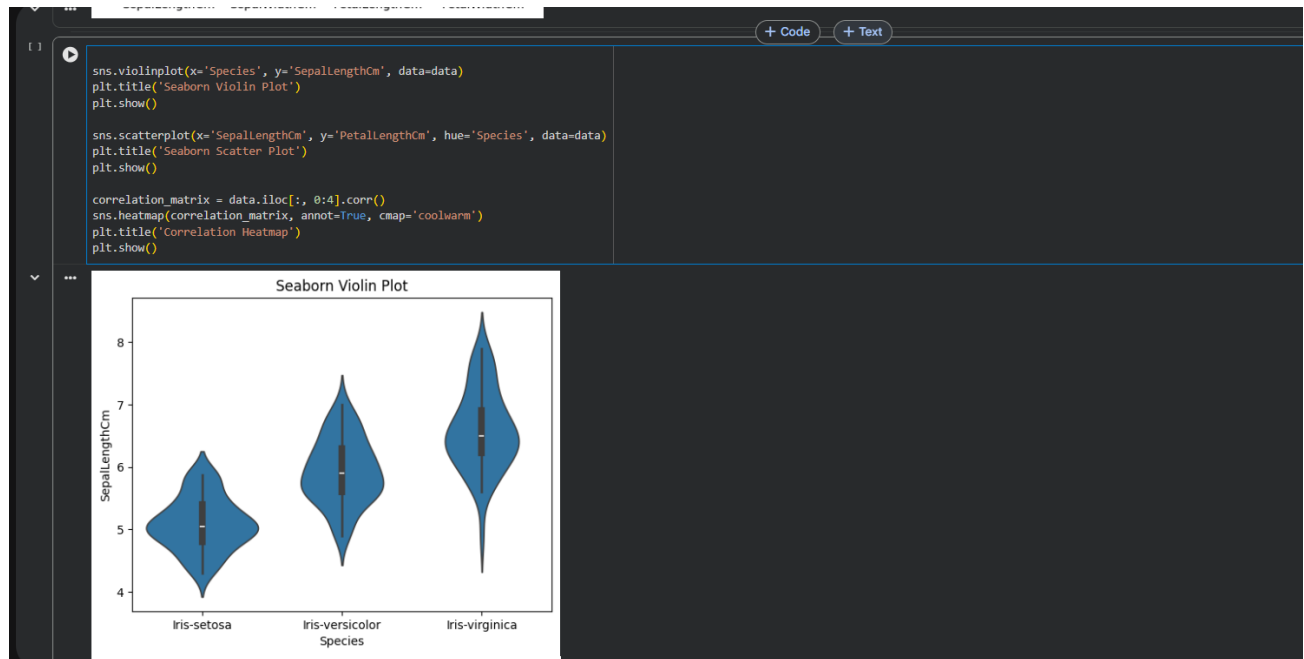
### # Scatterplot with Hue

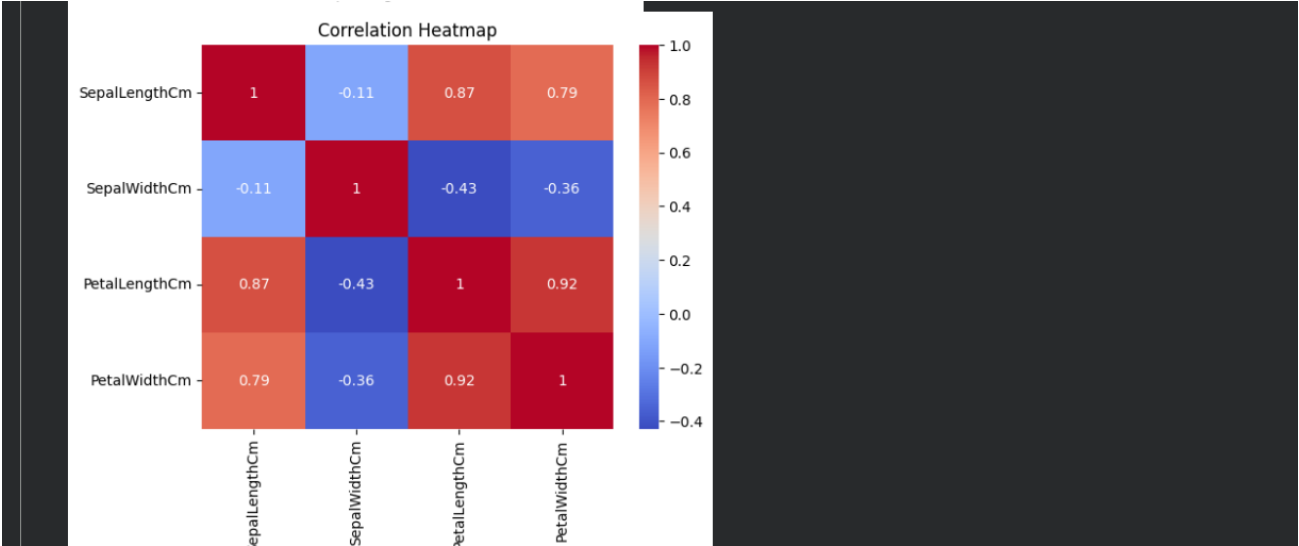
```
sns.scatterplot(x='SepalLengthCm', y='PetalLengthCm', hue='Species',
data=data)
plt.title('Seaborn Scatter Plot')
plt.show()
```

### # Correlation Heatmap

```
correlation_matrix = data.iloc[:, 0:4].corr()
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```

A violin plot illustrating the distribution of sepal length for each Iris species, showing density and statistical spread.







## Feature Engineering:-

- **Feature Creation:** Multiplying or adding existing columns to derive a brand new mathematical metric (like Area)
- **Binning :** Converting continuous numerical ranges into distinct text categories (e.g., Small, Medium, Large)
- **Label Encoding:** Converting text labels into numeric integers (e.g., 0, 1, 2) because machine learning models cannot read text

Code:

**# 1. Mathematical Derivation (Creating Petal Area)**

```
data['PetalAreaCm'] = data['PetalLengthCm'] * data['PetalWidthCm']
```

**# 2. Binning (Grouping continuous data into categories)**

```
bins = [0, 2.5, 5, 7]
```

```
labels = ['Small', 'Medium', 'Large']
```

```
data['PetalLengthCm_Group'] = pd.cut(data['PetalLengthCm'], bins=bins,  
labels=labels)
```

**# 3. Label Encoding (Converting string categories to numeric)**

```
le = LabelEncoder()
```

```
data['Species_encoded'] = le.fit_transform(data['Species'])
```

```
print("Final Dataset with Engineered Features:")
```

```
display(data.head())
```

\*\*\* Dataset with Engineered Features:

|   | SepallengthCm | SepalWidthCm | PetalLengthCm | PetalWidthCm | Species     | PetalAreaCm | PetalLengthCm_Group | Species_encoded |
|---|---------------|--------------|---------------|--------------|-------------|-------------|---------------------|-----------------|
| 0 | 5.871233      | 3.0          | 4.4           | 0.2          | Iris-setosa | 0.88        | Medium              | 0               |
| 1 | 5.871233      | 3.0          | 4.4           | 0.2          | Iris-setosa | 0.88        | Medium              | 0               |
| 2 | 5.871233      | 3.2          | 4.4           | 0.2          | Iris-setosa | 0.88        | Medium              | 0               |
| 3 | 5.871233      | 3.1          | 4.4           | 0.2          | Iris-setosa | 0.88        | Medium              | 0               |
| 4 | 5.000000      | 3.6          | 1.4           | 0.2          | Iris-setosa | 0.28        | Small               | 0               |